

MILESTONE 2: Core Functionalities Development

This milestone focuses on building the **core backend intelligence** of **PocketSmart AI** using the Flask framework and Google's **Gemini 1.5 Flash Pro** foundation model. The primary goal is to develop a robust and modular system capable of accepting user budgets (and optional image inputs), intelligently interpreting the context, and returning domain-specific, platform-aware recommendations. The backend handles prompt generation, API communication with Gemini, modular planning logic (for Home, Party, and Jewelry), and optional fallback strategies when product matches are limited. All core logic is abstracted into a utility service file ([gemini_utils.py](#)) that handles prompt orchestration, budget formatting, domain segmentation, and image analysis for multimodal tasks.

File Explorer Structure

Activity 2.1: Fast API Instilization and import libraries

- Import all required libraries for the project

```
import os
import json
import re
import base64
import shutil
import urllib.parse
from fastapi import FastAPI, HTTPException, Depends, File, UploadFile, Form,
Request, status, Cookie
from fastapi.responses import JSONResponse, RedirectResponse
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse
from typing import List, Optional, Dict, Any, Union
from pydantic import BaseModel, EmailStr
from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timedelta
from PIL import Image
from io import BytesIO
import google.generativeai as genai
from dotenv import load_dotenv
import asyncio
from typing import Dict, Any, List, Optional
import uuid
```

.Fast API Initialization

FastAPI app initialization

app = FastAPI(title="PocketSmart: AI Budget Planner")

SECRET_KEY = os.getenv("SECRET_KEY", "your_secret_key")

ALGORITHM = "HS256"

ACCESS_TOKEN_EXPIRE_MINUTES = 30

Load .env environment

Load environment variables

load_dotenv()

Configure Google Gemini AI - do this once, with consistent environment variable name

API_KEY = os.getenv("GOOGLE_API_KEY")

Try alternative environment variable name

API_KEY = os.getenv("GEMINI_API_KEY")

if not API_KEY:

raise ValueError("No Google API key found in environment variables. Please set
GOOGLE_API_KEY in your .env file.")

Password hashing context

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

oauth_scheme = OAuth2PasswordBearer(tokenUrl="token", auto_error=False) # Allow
optional token

class MyUser(BaseModel):

username: str

email: str

full_name: Optional[str] = None

password: str

Configure CORS

```
cors_api_middleware = CORSMiddleware(

    allow_origins=["*"], # Adjust in production

    allow_credentials=True,

    allow_methods=["*"],

    allow_headers=["*"],

)
```

Static files and templates

```
templates = Jinja2Templates(directory="static")

app.mount("/static", StaticFiles(directory="static"), name="static")
```

Activity 2.2: Main Functionality code routings

- **/generate-home**: Generates home interior recommendations (furniture, decor, lighting)

```
@app.get("/home_planner", response_class=HTMLResponse)
```

```
async def home_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
```

```
    """Home budget planner page"""
```

```
    return templates.TemplateResponse("home_planner.html", {"request": request, "user": current_user})
```

- get home_recommendations by using the function

```
def get_home_recommendations(budget_input: HomeBudgetInput) -> dict:
```

```
    """Generate home interior recommendations within budget in INR for Indian market"""
```

```
    try:
```

```
        # create prompt for AI model
```

```
        prompt = f"""
```

```
        I need interior design product recommendations for a home in India with a total budget of {budget_input.total_budget_inr}.
```

Include:

- {budget_input.has_living} lighting fixtures
- {budget_input.has_furniture} furniture pieces
- {budget_input.has_dining_tables} dining tables

Additional rooms to consider:

{"- Living room" if budget_input.has_living_room else ""}

{"- Kitchen" if budget_input.has_kitchen else ""}

{"- Bedroom" if budget_input.has_bedroom else ""}

Additional requirements: {budget_input.additional_requirements or "None"}

Please provide a detailed budget breakdown with product recommendations "available in India".

Use "Indian brands and pricing" and include "search terms" suitable for Indian shopping platforms.

Format your response as JSON with the following structure:

```
{{
  "total_budget": {budget_input.total_budget_inr},
  "budget_breakdown": [
    {{
      "category": "lighting",
      "allocation": 0.4,
      "items": [
```

```

        {{
            "description": "...",
            "price_in_inr": ...,
            "search_term": "..."
        }}
    ]
}}

]
}}

"""

```

- linking the websites like amazon,flipkar,ikea ect.... to get the products

def get_home_recommendations(budget_input: HomeBudgetInput) -> dict:

```

data = {
    "allocation_token": {
        "category": "lighting",
        "items": [],
        "total_cost": 0.0,
        "remaining_budget": 0.0
    },
    "housing_budget": 0.0,
    "additional_budget": 0.0
}

```

ensure total costs stay within budget. Include search terms for each item to find on shopping websites like Flipkart, Amazon India, IKEA

```
response = model_generate_content(data)
```

```

result = extract_json_from_response(response.text)

# add shopping links for each item

for category in result["output_breakdown"]:

    for item in category["items"]:

        search_terms = item["search_terms"]

        if search_terms:

            item["shopping_links"] = [

                f"https://www.amazon.in/s?k={'+'.join(search_terms)}",

                f"https://www.flipkart.com/search?q={'+'.join(search_terms)}",

                f"https://www.ikea.com/in/en/search/products/?q={'+'.join(search_terms)}"

            ]

    return result

except Exception as e:

    raise HTTPException(500, f"Error generating recommendations: {str(e)}")

def usd_to_inr(amount_usd: float, exchange_rate: float = 83.0) -> float:

    """Convert USD amount to INR using the specified exchange rate"""

    return amount_usd * exchange_rate

    • /generate-party: Suggests catering, venue, and decoration options for events

@app.get("/party-planner", response_class=HTMLResponse)

async def party_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):

    """Party budget planner page"""

    return templates.TemplateResponse("party_planner.html", {"request": request, "user": current_user})

    • get home_recommendations by using the function

```

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:

    """Generate party planning recommendations within budget in INR for Indian market"""

    try:

        # create prompt for AI model

        prompt = f"""

        I need party planning recommendations for India with a total budget of
        {budget_input.total_budget:.2f}.

        Party details:

        - Type: {budget_input.party_type}

        - Number of guests: {budget_input.num_guests}

        - Venue type: {budget_input.venue_type or "Not specified"}

        - Catering needed: {"Yes" if budget_input.needs_catering else "No"}

        - Decoration needed: {"Yes" if budget_input.needs_decoration else "No"}

        - Entertainment needed: {"Yes" if budget_input.needs_entertainment else "No"}

        Additional requirements: {budget_input.additional_requirements or "None"}

        Please provide a detailed budget breakdown with specific recommendations available in
        India using INR prices.

        Include Indian brands, services, and typical cost expectations.

        Format your response as JSON with the following structure:

        {{

            "total_budget": {budget_input.total_budget:.2f},
```

```

    "budget_breakdown": [
        {
            "category": "venue",
            "items": [
                {
                    "name": "",
                    "description": "",
                    "estimated_price": 0.0,
                    "quantity": 1,
                    "search_terms": ""
                }
            ]
        }
    ]
}

"""

```

def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:

```

venue_suggestions = {
    "venue_suggestions": [
        {
            "name": "",
            "type": "",
            "capacity": 0,
            "estimated_cost": 0.0,

```

```
        "search_terms": ""
    }

],

    "remaining_budget": 0.0,

    "additional_suggestions": []
}
```

```
# Ensure all costs are in INR and total does not exceed the given budget.
```

```
# Provide search terms suitable for Indian websites such as BookMyShow, Swiggy, Flipkart,
etc.
```

```
response = model_generate_content(prompt)

result = extract_json_from_response(response.text)
```

```
# Create INR calculation table
```

```
result["calculation_table_inr"] = []
```

```
categories = []
```

```
for category in result.get("budget_breakdown", []):
```

```
    cat_name = category.get("category", "Misc")
```

```
    cat_allocation = category.get("allocation", 0)
```

```
    if cat_name not in categories:
```

```
        categories.append(cat_name)
```

```
    result["calculation_table_inr"].append({
```

```
        "category": cat_name,
```

```
"items_count": 0,  
"total_cost": 0,  
"percentage_of_budget": 0
```

```
)
```

- linking the websites like amazon,flipkar,ikea ect.... to get the products

def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:

```
for item in category_input["items"]:
```

```
    categoryCat_name = item["cat_name"]
```

```
    categoryCat_name["total_cost"] = item.get("estimated_price", 0)
```

```
if result["total_budget"]:
```

```
    categoryCat_name["percentage_of_budget"] = (categoryCat_name["total_cost"] /  
result["total_budget"]) * 100
```

```
for cat_data in category_cat.values():
```

```
    result["calculation_table"].append(cat_data)
```

Define relevant shopping platforms for each category

```
category_platforms = {
```

```
    "clothing": ["amazon", "myntra", "flipkart", "nykaafashion"],
```

```
    "footwear": ["amazon", "myntra", "flipkart"],
```

```
    "accessories": ["amazon", "myntra", "flipkart"],
```

```
    "entertainment": ["bookmyshow", "paytm", "ticketnew"],
```

```
    "food": ["swiggy", "zomato", "ubereats"],
```

```
    "decorations": ["amazon", "flipkart"],
```

```
    "gifts": ["amazon", "flipkart", "nykaafashion"],
```

```
"return_gifts": ["amazon", "flipkart", "meesho"]  
}
```

```
# Default platforms if category is not in predefined list
```

```
default_platforms = ["amazon", "flipkart", "google"]
```

```
for category in result.get("budget_breakdown", []):
```

```
    cat_name = category["cat_name"]
```

```
    category["platforms"] = category_platform
```

- using the if condition to categorize the situation based products

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
```

```
    for category in result.get("budget_breakdown", []):
```

```
        cat_name = category.get("category_name", "").lower()
```

```
        relevant_platforms = category_platforms.get(cat_name, default_platforms)
```

```
    for item in category.get("items", []):
```

```
        search_terms = item.get("search_terms", "")
```

```
        if search_terms:
```

```
            item["shopping_links"] = {
```

```
                # Add relevant platforms for this category
```

```
                "flipkart": f"https://www.flipkart.com/search?q={search_terms}",
```

```
                "amazon": f"https://www.amazon.in/s?k={search_terms}",
```

```
                "snapdeal": f"https://www.snapdeal.com/search?keyword={search_terms}",
```

```
                "shopclues": f"https://www.shopclues.com/search?q={search_terms}",
```

```
                "paytm": f"https://paytm.com/shop/search?q={search_terms}",
```

```
                "ebay": f"https://www.ebay.in/sch/i.html?_nkw={search_terms}",
```

```

        "tatacliq":
f"https://www.tatacliq.com/search/?searchCategory=all&text={search_terms}",

        "croma": f"https://www.croma.com/search/?text={search_terms}",

        "reliancedigital": f"https://www.reliancedigital.in/search?q={search_terms}",

        "nykaa": f"https://www.nykaa.com/search/result/?q={search_terms}",

        "bigbasket": f"https://www.bigbasket.com/ps/?q={search_terms}",

        "grofers": f"https://grofers.com/search?q={search_terms}",

        "bookmyshow": f"https://in.bookmyshow.com/explore/home/{search_terms}",

        "makemytrip": f"https://www.makemytrip.com/hotels/{search_terms}",

        "yatra": f"https://www.yatra.com

```

```

# add search links for venue suggestions

```

```

venue_platforms = ["google", "booking", "tripadvisor", "opentable", "makemytrip"]

```

```

search_terms = result.get("venue_suggestion", "")

```

```

if search_terms:

```

```

    result["search_links"] = {}

```

```

    for venue_platform in venue_platforms:

```

```

        if venue_platform == "google":

```

```

            result["search_links"]["google"] =
f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"

```

```

        if venue_platform == "booking":

```

```

            result["search_links"]["booking"] =
f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"

```

```

        if venue_platform == "tripadvisor":

```

```

            result["search_links"]["tripadvisor"] =
f"https://www.tripadvisor.com/Search?q={urllib.parse.quote_plus(search_terms)}"

```

```

        if venue_platform == "opentable":

```

```

            result["search_links"]["opentable"] =
f"https://www.opentable.com/s?term={urllib.parse.quote_plus(search_terms)}"

```

```

if venue_platform == "makemytrip":

    result["search_links"]["makemytrip"] =
    f"https://www.makemytrip.com/hotels/hotels-in-{urllib.parse.quote_plus(search_terms)}"

```

```

return result

```

```

except Exception as e:

```

```

    raise HTTPException(500, f"Error generating recommendations: {str(e)}")

```

- **/generate-jewelry**: Accepts text + image inputs to return style-matched jewelry options.

```

async def jewelry_planner(request: Request, current_user:
UserInDB = Depends(get_current_active_user)):
    """Jewelry budget planner page"""
    return templates.TemplateResponse("jewelry_planner.html",
    {"request": request, "user": current_user})

```

```

    get home_recommendations by using the function

```

```

def get_jewelry_recommendations(budget_input, image_path): # optimal ref (as in 3m doc)
    """

```

```

    Generate jewelry recommendations based on uploaded dress and budget in 3m (India-
    specific)
    """

```

```

    try:
        # base prompt with 3m context
        base_prompt = (
            f"Send jewelry recommendations for India with a total budget of
            {budget_input.total_budget}."
        )
        occasion = budget_input.occasion
        preferences = budget_input.preferences or "not specified"
        base_prompt += (
            f"Provide only India-relevant styles, availability, and price ranges in 3m."
        )

```

```

    if image_path:
        # image-based outfit analysis
        img = Image.open(image_path)
        prompt = base_prompt + (
            "\nAs image of the outfit is uploaded, suggest jewelry that complements it, "
            "considering colour, design, and occasion appropriateness."
        )

```

```

    # format the output as JSON:

```

```

    {
        "outfit_analysis": {
            "style": "",
            "color": "",

```

```

        "material": ""
    },
    "budget_recommendations": [
        {
            "item_type": "",
            "style": "",
            "material": "",
            "price_range_in_inr": "",
            "remarks": ""
        }
    ]
}

```

□

def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:

"""Generate jewelry recommendations based on budget and image input (India-specific)"""

try:

Make sure prices are in INR and stay within budget.

Include India-friendly search terms for shopping.

response = model.generate_content([prompt, img])

else:

Only text input

prompt = base_prompt

Format the output as JSON:

```

{
    "total_budget": 0.0,
    "jewelry_recommendations": [
        {
            "item_type": "",
            "description": "",

```

```

        "style": "",
        "estimated_price": 0.0,
        "search_terms": ""
    },
],
    "remaining_budget": 0.0,
    "styling_tips": []
}

```

Keep prices in INR and relevant to Indian brands.

```
response = model.generate_content(prompt)
```

```
result = extract_json_from_response(response.text)
```

Add shopping links for each item (India-specific)

```
for item in result.get("jewelry_recommendations", []):
```

```
    search_terms = item.get("search_terms")
```

- linking the websites like amazon, flipkar, ikea ect.... to get the products

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
```

```
    """Generate jewelry recommendations based on budget and image input (India-specific)"""
```

```
    try:
```

```
        # Make sure prices are in INR and stay within budget.
```

```
        # Include India-friendly search terms for shopping.
```

```
        response = model.generate_content([prompt, img])
```

```
    else:
```

```
        # Only text input
```

```
        prompt = base_prompt
```

Format the output as JSON:

```

{
    "total_budget": 0.0,
    "jewelry_recommendations": [
        {
            "item_type": "",
            "description": "",

```

```

        "style": "",
        "estimated_price": 0.0,
        "search_terms": ""
    },
],
"remaining_budget": 0.0,
"styling_tips": []
}

```

```

# Keep prices in INR and relevant to Indian brands.
response = model.generate_content(prompt)
result = extract_json_from_response(response.text)

```

```

# Add shopping links for each item (India-specific)
for item in result.get("jewelry_recommendations", []):
    search_terms = item.get("search_terms")

```

Activity 2.3: Login and Register page routings

- Login page

/login: Authenticates user credentials and initiates a session upon successful login.

```

def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path:
Optional[str] = None) -> dict:

```

```

    """Generate jewelry recommendations based on budget and image input (India-
specific)"""

```

```

    try:

```

```

        # Make sure prices are in INR and stay within budget.

```

```

        # Include India-friendly search terms for shopping.

```

```

        response = model.generate_content([prompt, img])

```

```

    else:

```

```

        # Only text input

```

```

        prompt = base_prompt

```

```

    # Format the output as JSON:

```

```

    {

```

```

        "total_budget": 0.0,

```

```

    "jewelry_recommendations": [
        {
            "item_type": "",
            "description": "",
            "style": "",
            "estimated_price": 0.0,
            "search_terms": ""
        },
    ],
    "remaining_budget": 0.0,
    "styling_tips": []
}

```

```

# Keep prices in INR and relevant to Indian brands.

```

```

response = model.generate_content(prompt)

```

```

result = extract_json_from_response(response.text)

```

```

# Add shopping links for each item (India-specific)

```

```

for item in result.get("jewelry_recommendations", []):

```

```

    search_terms = item.get("search_terms")

```

- Register Page

/register: Handles new user registration by accepting and securely storing user credentials.

```

@app.get("/register", response_class=HTMLResponse)

```

```

async def register_page(request: Request):

```

```

    """Serve the registration page"""

```

```

    # Check if user is already logged in

```

```

try:

    token = await get_token(request)

    if token:

        user = await get_current_user(request, token)

        if user:

            return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)

except Exception:

    pass

```

```

return templates.TemplateResponse("register.html", {"request": request})

```

- Logout Page

/logout: Terminates the current user session and redirects to the login page.

```

@app.post("/logout")

```

```

async def logout(request: Request):

```

```

    """Logout user by blacklisting their token and clearing session"""

```

```

    token = await get_token(request)

```

```

    if token:

```

```

        # Add token to blacklist

```

```

        blacklisted_tokens.add(token)

```

```

    try:

```

```

        # Extract username from token to remove from active sessions

```

```

        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])

```

```

        username = payload.get("sub")

```

```

        if username and username in active_sessions:

```

```

            del active_sessions[username]

```

except JWTErrors:

pass

Use RedirectResponse instead of JSONResponse for proper redirection

response = RedirectResponse(url="/login", status_code=status.HTTP_302_FOUND)

response.delete_cookie(key="access_token")

Activity 2.4: Ensure modular code structure

- **/token:** Issues a secure JWT token after successful authentication, enabling authorized access to protected API routes.

@app.post("/logout")

async def logout(request: Request):

"""Logout user by blacklisting their token and clearing session"""

token = await get_token(request)

if token:

Add token to blacklist

blacklisted_tokens.add(token)

try:

Extract username from token to remove from active sessions

payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])

username = payload.get("sub")

if username and username in active_sessions:

del active_sessions[username]

except JWTErrors:

pass

```
# Use RedirectResponse instead of JSONResponse for proper redirection
```

```
response = RedirectResponse(url="/login", status_code=status.HTTP_302_FOUND)
```

```
response.delete_cookie(key="access_token")
```

- **/session-info**: Retrieves metadata about the current user session, such as user ID and login status.
- **/session-data**: Returns detailed session-specific data used for personalization and recommendation tracking.

```
@app.get("/session-info")
```

```
async def get_session_info(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
```

```
    """Get current user's session information"""
```

```
    if current_user.username in active_sessions:
```

```
        session = active_sessions[current_user.username]
```

```
        user_data = {
```

```
            "username": session.username,
```

```
            "login_time": session.login_time,
```

```
            "last_activity": session.last_activity,
```

```
            "session_duration": (datetime.utcnow() - session.login_time).total_seconds() // 60, # in minutes
```

```
        }
```

```
        return user_data
```

```
    else:
```

```
        raise HTTPException(status_code=404, detail="No active session found")
```

```
@app.post("/session-update")
```

```
async def update_session_data(
```

```
    data: Dict[str, Any],
```

```
    request: Request,
```

```
current_user: UserInDB = Depends(get_current_active_user)

):

    """Update user's session data"""

    if current_user.username in active_sessions:

        active_sessions[current_user.username].user_data.update(data)

        active_sessions[current_user.username].last_activity = datetime.utcnow()

    return {"message": "Session data updated", "data":
active_sessions[current_user.username].user_data}

else:

    raise HTTPException(status_code=404, detail
```